ASSIGNMENT 3 – TASK 5

PAGE-REPLACEMENT TECHNIQUES

Q5 – Page-Replacement Techniques | 8 Marks | Bloom's Level: K5 – Evaluate

1. Introduction and Problem Context

The assignment describes a web server that keeps frequently used data pages in memory. When all frames are occupied, the operating system must select a victim page. A poor choice increases page faults and response time. This report implements FIFO, LRU and Optimal replacement, adds Clock (second-chance) for comparison, verifies the supplied reference string, demonstrates Belady's anomaly, and measures page-fault counts and execution time for generated workloads.

2. Implementation of Page-Replacement Algorithms

FIFO: The page that entered memory earliest is replaced. A circular pointer identifies the next frame to replace. FIFO is simple and does not use recent-access information.

LRU: The least recently used page is replaced. The completed TODO 1 maintains the latest reference index in last[]. On a hit, last[k] is updated to the current reference index. On a miss, a free frame is used if available; otherwise, the frame with the smallest last[] value is the victim.

Optimal: The page whose next use is farthest in the future is replaced; a page never used again is preferred. It is the theoretical minimum-fault policy for a known reference string, but a running operating system cannot know future references. It is included as an offline benchmark. The large-workload simulator precomputes next-use positions so the offline simulation remains efficient.

Clock / second chance: Each frame has a reference bit and a circular hand. A hit sets the bit. When replacement is needed, the hand clears bits set to 1 and advances; the first frame with bit 0 is replaced. The new page receives reference bit 1. Clock approximates recency without maintaining an exact LRU order on every access.

3. Completed TODO 1 – LRU Replacement

The missing block in the supplied lru() function is completed as follows:

```
int victim;

if (used < frames) {
    victim = used++;
    f[victim] = ref[i];
} else {
    victim = 0;
    for (int j = 1; j < frames; j++) {
        if (last[j] < last[victim])
            victim = j;
    }
    f[victim] = ref[i];
}

last[victim] = i;
```

The complete implementation is in page_replacement.c. The update last[victim] = i is required both when filling a free frame and when replacing a victim, so that the next replacement decision has the correct recency information.

4. Given Reference String – Required Output

The supplied 20-reference string is:

```
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
```

Number of frames	FIFO faults	LRU faults	Optimal faults
3	15	12	9
4	10	8	8

The program reproduces the expected output printed in the assignment. Optimal has the fewest faults because it uses future knowledge. LRU improves on FIFO for this supplied sequence.

4.1 Hand verification: LRU with 3 frames

For each reference, a hit leaves the frame set unchanged and updates the most-recent-use time. A fault fills a free frame or replaces the page with the smallest last-use index.

Step	Page	Frames after reference	Result
1	7	{7}	Fault
2	0	{7 0}	Fault
3	1	{7 0 1}	Fault
4	2	{2 0 1}	Fault
5	0	{2 0 1}	Hit
6	3	{2 0 3}	Fault
7	0	{2 0 3}	Hit
8	4	{4 0 3}	Fault
9	2	{4 0 2}	Fault
10	3	{4 3 2}	Fault
11	0	{0 3 2}	Fault
12	3	{0 3 2}	Hit
13	2	{0 3 2}	Hit
14	1	{1 3 2}	Fault
15	2	{1 3 2}	Hit
16	0	{1 0 2}	Fault
17	1	{1 0 2}	Hit
18	7	{1 0 7}	Fault
19	0	{1 0 7}	Hit
20	1	{1 0 7}	Hit

Counting the Fault rows gives 12 page faults, matching the C program and the expected assignment result for LRU with 3 frames.

5. Belady’s Anomaly with FIFO

The second supplied reference string is:

```
1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
```

Step	Page	FIFO state (3 frames)	Result	FIFO state (4 frames)	Result
1	1	{1}	Fault	{1}	Fault
2	2	{1 2}	Fault	{1 2}	Fault
3	3	{1 2 3}	Fault	{1 2 3}	Fault
4	4	{4 2 3}	Fault	{1 2 3 4}	Fault
5	1	{4 1 3}	Fault	{1 2 3 4}	Hit
6	2	{4 1 2}	Fault	{1 2 3 4}	Hit
7	5	{5 1 2}	Fault	{5 2 3 4}	Fault
8	1	{5 1 2}	Hit	{5 1 3 4}	Fault
9	2	{5 1 2}	Hit	{5 1 2 4}	Fault
10	3	{5 3 2}	Fault	{5 1 2 3}	Fault
11	4	{5 3 4}	Fault	{4 1 2 3}	Fault
12	5	{5 3 4}	Hit	{4 5 2 3}	Fault

FIFO incurs 9 faults with 3 frames but 10 faults with 4 frames. With 4 frames, FIFO initially retains pages 1, 2, 3 and 4. After page 5 arrives, the oldest pages are repeatedly removed in an order that causes the tail of the reference string to fault on pages 1, 2, 3, 4 and 5. With 3 frames, the earlier replacements leave page 5 resident for the final reference, avoiding one fault. This counter-intuitive increase is Belady's anomaly.

6. Why LRU Cannot Suffer Belady's Anomaly

LRU has the stack property. For a fixed reference prefix, the set of pages resident with k frames is always a subset of the pages resident with k+1 frames. Increasing the number of frames cannot remove a page that would have been resident with the smaller allocation. Therefore the page-fault count under LRU is non-increasing as the number of frames increases. FIFO does not have this inclusion property because its queue order changes with the number of frames.

7. Hardware Support and Complexity

Algorithm	Per-reference work in this implementation	State / practical consideration
FIFO	Linear frame search O(F); pointer update O(1).	Small state: frame array and circular pointer. No recency tracking.
LRU	Linear frame search O(F); on a hit update timestamp; on a full miss scan F timestamps to select the oldest.	One timestamp per frame in this simulation. Exact LRU requires access-order updates; hardware support for exact ordering is costly. A linked list plus lookup can make software updates O(1), but each access still changes order.
Optimal	Offline next-use preprocessing O(N + P) in this benchmark, then O(F) lookup/victim scan per reference.	Requires future references. Not implementable as an online policy.
Clock	Linear frame search O(F); reference-bit set on hit; a fault may scan up to O(F) frames.	One reference bit per frame plus a hand pointer. Hardware accessed/reference bits make the policy inexpensive compared with exact LRU.

Here F is the number of frames, N is the number of references and P is the page-number domain size. The C implementations use a linear find() function, so their frame lookup costs O(F). In a production OS the address translation path is accelerated by page tables and the TLB; the table compares the

replacement-policy bookkeeping itself. LRU typically needs a timestamp or an updateable recency list, whereas Clock can use a hardware-maintained reference bit and only scans on a replacement fault.

8. Experimental Workloads

The program generates two deterministic reference strings of 100,000 references each. The strong-locality workload sends 90% of references to a hot set of 8 pages and 10% to a broader 64-page range. The random workload selects uniformly from 64 pages. A fixed pseudo-random seed makes the sequences reproducible. All four algorithms are tested with 3–10 frames.

8.1 Page faults – strong locality

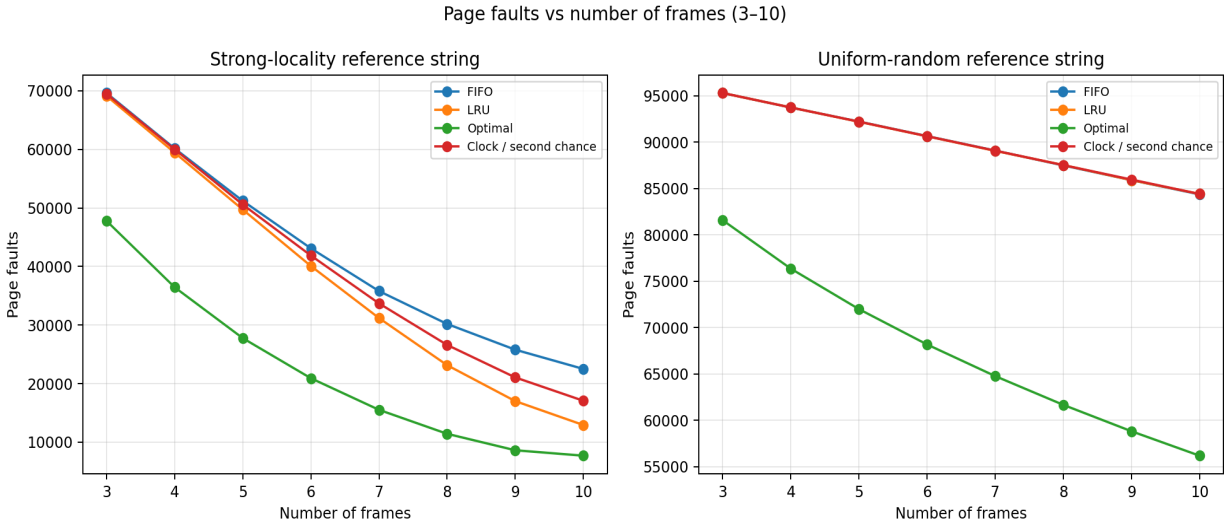
Frames	FIFO	LRU	Optimal	Clock
3	69630	69127	47790	69480
4	60156	59443	36452	60016
5	51206	49716	27762	50578
6	43101	40081	20916	41870
7	35803	31193	15520	33698
8	30184	23147	11425	26604
9	25790	17008	8615	21056
10	22504	12927	7687	17068

8.2 Page faults – random access

Frames	FIFO	LRU	Optimal	Clock
3	95299	95297	81575	95304
4	93738	93742	76348	93737
5	92221	92199	71983	92211
6	90657	90636	68169	90638
7	89089	89059	64757	89082
8	87489	87530	61652	87523
9	85900	85900	58806	85961
10	84365	84413	56168	84405

Optimal has the lowest fault count in both generated workloads, as expected from its future knowledge. LRU performs better than FIFO and Clock in the stronger-locality workload at higher frame counts. For uniformly random accesses over 64 pages with only 3–10 frames, all practical policies experience many faults because the resident set is much smaller than the reference domain.

9. Page-Fault Plot



The two panels plot page faults against frame count for FIFO, LRU, Optimal and Clock. Increasing frames reduces or preserves faults for these simulations. Optimal remains the lower bound. The locality workload exposes the advantage of recency-sensitive policies; the random workload has weak locality and therefore high fault counts for all practical algorithms.

10. Runtime Measurement – 100,000 References

Each algorithm was run three times per workload and frame count. The median elapsed time is recorded in runtime_100k.csv. The table below gives the mean of the eight per-frame medians (3–10 frames), in milliseconds, for the reference execution environment. The full frame-by-frame timing data remains in the CSV.

Workload	Algorithm	Mean ms (frames 3–10)	At 10 frames (ms)
Strong locality	FIFO	0.735	0.837
Strong locality	LRU	0.984	1.270
Strong locality	Optimal	1.135	1.127
Strong locality	Clock	1.147	1.242
Random	FIFO	0.464	0.592
Random	LRU	1.052	1.714
Random	Optimal	1.455	1.842
Random	Clock	1.119	1.474

These timing values were captured in the artifact-preparation environment (GCC 14.2.0, Linux x86_64, 5 reported CPUs). They are reference measurements, not measurements from the student's computer. Runtime varies with the processor, compiler, operating system and system load. Rerun ./run_task5.sh and use the generated runtime_100k.csv if local measurements are required.

11. Clock Compared with LRU

For the strong-locality simulation, Clock is within 0.51% of LRU at 3 frames and within 0.96% at 4 frames, but is 32.03% higher at 10 frames. Across 3–10 frames the mean absolute Clock/LRU fault-count gap is about 10.81%. For the random workload, Clock and LRU are nearly identical in this run (mean absolute gap about 0.018%), but both have high fault counts; the similarity reflects the weak locality of the workload rather than strong replacement quality.

Clock uses one reference bit per frame and a hand pointer, while exact LRU tracks recency on every hit. Clock can therefore reduce policy bookkeeping and exploit hardware reference bits. The trade-off is that its victim may be older or newer than the exact least-recently-used page, so the fault count can be higher.

12. Engineering Decision for the Web Server

For the generated strong-locality web-server workload, LRU is the recommended policy when reducing page faults is the primary objective and recency bookkeeping is affordable. At 10 frames, the simulation records 12,927 LRU faults versus 17,068 for Clock and 22,504 for FIFO. Optimal records 7,687 faults but cannot be used by a live server because future references are unknown.

Clock is a practical alternative where low bookkeeping overhead and hardware reference-bit support are more important than achieving exact LRU. It should be validated against the server's real request trace, because the simulated Clock/LRU gap grows at higher frame counts in the strong-locality workload. For this assignment's measured locality workload, LRU offers the stronger fault reduction and is the final recommendation.

13. Conclusion

The completed LRU code reproduces the supplied expected results and the FIFO Belady-anomaly example. LRU's stack property guarantees that adding frames cannot increase its fault count. Optimal provides a theoretical lower bound but needs future-reference knowledge. Clock approximates LRU using one reference bit per frame and a circular hand. The 100,000-reference experiments show that locality strongly affects the choice: LRU reduces faults for the generated strong-locality workload, while random access produces high fault counts for every practical policy. Based on those measurements, LRU is recommended for the simulated web server, with Clock as a practical low-bookkeeping alternative.

14. Execution Procedure and Evidence

1. Place page_replacement.c, plot_page_faults.py and run_task5.sh in one Task05 folder.
2. Open Ubuntu/WSL in that folder.
3. Run chmod +x run_task5.sh.
4. Run ./run_task5.sh.
5. Verify the supplied 3-frame/4-frame output and Belady anomaly result.
6. Review faults_locality.csv, faults_random.csv and runtime_100k.csv.
7. Review page_fault_comparison.png.
8. Capture screenshots of terminal output and include generated evidence in the Task05 folder.

Evidence checklist: working FIFO/LRU/Optimal code; exact expected output; hand-verified LRU trace; FIFO Belady trace and explanation; locality/random 3–10 frame counts; page-fault graph; runtime measurements for 100,000 references; Clock comparison; and measurement-based engineering recommendation.

Source alignment: Q5 – Page-Replacement Techniques, 8 Marks, K5 – Evaluate, pages 27–28 of the supplied Assignment 3 question paper.